IBM Data Engineering Course | Coursera

Data Wherehouse fundamentals

Summary

Module one

Data Wherehouse, data marts, and data leaks

An Introduction to Data Warehouses, Data Marts, and Data Lakes

📘 Lesson 1: Data Warehouse Overview

🧠 Objectives

By the end of this lesson, you will be able to:

- Define what a data warehouse is
- Identify real-world use cases of data warehouses
- List key benefits of using a data warehouse

🛠 Key Concepts

🔹 **What is a Data Warehouse?**

- A **data warehouse** is a **centralized system** that aggregates data from multiple sources into a consistent store.
- Supports various **data analytics** operations like:
  - **Data mining**
  - **AI and ML applications**
  - **OLAP (Online Analytical Processing)**
  - **ETL for front-end reporting**

🔹 **Types of Data Warehouses**

- **Traditional / On-premises**: Deployed on mainframes or local servers (Unix, Windows, Linux).
- **Appliances**: Bundled hardware + software solutions for better performance (popular in 2000s).
- **Cloud Data Warehouses (CDW)**: Scalable, cloud-hosted, subscription-based (no hardware ownership required).

🔹 **Use Cases by Industry**

- **Retail / E-commerce**: Sales performance, personalized recommendations via ML
- **Healthcare**: AI-assisted diagnosis and treatment
- **Transportation**: Route optimization and staffing
- **Banking / FinTech**: Fraud detection, risk evaluation, cross-selling
- **Social Media**: Sentiment analysis, product forecasting
- **Government**: Program analysis, policy planning

🔹 **Benefits of Data Warehousing**

- **Data Centralization**: Integrates various formats (transactional systems, flat files, etc.)
- **Data Quality**: Cleansing, deduplication, and standardization
- **Single Source of Truth**: Consistent, trustworthy analytics foundation
- **Performance**: Separation of analytics from transactional operations speeds up insights
- **Advanced Analytics Support**: Enables large-scale BI, AI, ML, and data mining

---

🧪 Example Use

A healthcare provider pulls patient data from multiple hospital systems into a data warehouse. With AI tools, it identifies high-risk patients and recommends early interventions — improving treatment outcomes.

🧩 Key Takeaways

- A **data warehouse** consolidates and cleans data to support reliable analytics and BI.
- It enables faster, smarter decisions across diverse industries using tools like **OLAP**, **AI/ML**, and **data mining**.
- **Cloud Data Warehouses** offer flexibility and scalability, aligning with modern data demands.

📘 Lesson 2: Popular Data Warehouse Systems

🧠 Objectives

By the end of this lesson, you will be able to:

- Categorize data warehouse systems based on deployment models
- List popular vendors and their offerings in the data warehousing landscape

---

🛠 Key Concepts

🔹 Deployment Categories of Data Warehouse Systems

1. **Appliances**:
   - Pre-integrated bundles of hardware and software
   - High performance with lower maintenance overhead
   - Example: Oracle Exadata, IBM Netezza
2. **Cloud-Based Solutions**:
   - Fully managed, scalable services
   - Pay-as-you-go models with high availability
   - Example: Amazon Redshift, Snowflake, Google BigQuery
3. **Hybrid (On-Premises + Cloud)**:
   - Software available both on-prem and in cloud environments
   - Flexible deployment, supports modern hybrid IT infrastructures
   - Example: Microsoft Azure Synapse, IBM Db2 Warehouse, Teradata Vantage

---

🧩 Vendor Highlights

⚙️ Appliance-Based Systems

- **Oracle Exadata**:
  - Supports OLTP, analytics, in-memory, and mixed workloads
  - Deployable on-premises or via Oracle Cloud
- **IBM Netezza**:
  - Strong in ML and data science workloads
  - Deployable across IBM Cloud, AWS, Azure, and private clouds

☁️ Cloud-Based Warehouses

- **Amazon Redshift**:
  - Built for AWS
  - Offers ML, encryption, and auto-optimization
  - Graph optimization for smart data organization
- **Snowflake**:
  - Multi-cloud compliance (GDPR, CCPA, FedRAMP Moderate)
  - Always-on encryption for data at rest and in motion
- **Google BigQuery**:
  - Petabyte-scale, sub-second queries
  - Uptime of 99.99%, supports real-time analytics and high concurrency

🌐 Hybrid / Multi-Cloud Platforms

- **Microsoft Azure Synapse Analytics**:

    - Visual ETL/ELT tools
    - Supports T-SQL, Spark, Python, Scala, .NET
    - Works for both data lakes and data warehouses
- **Teradata Vantage**:
    - Unified platform for lakes, warehouses, and other data sources
    - High concurrency, adaptive optimization, enterprise analytics focus
- **IBM Db2 Warehouse**:
    - Containerized, scalable, petaflop-speed performance
    - Flexible movement across on-prem, private, and public clouds
- **Vertica**:
    - Multi-cloud and on-prem support
    - High data transfer speed, elastic resources, fault-tolerant with Eon mode
- **Oracle Autonomous Data Warehouse**:
    - Self-managing and self-securing
    - Automated encryption, patching, and threat detection

🧪 Example Use

A company uses **Snowflake** across AWS and Azure to ensure data governance compliance while running AI models for customer segmentation, benefiting from secure and scalable analytics with minimal overhead.

🧩 Key Takeaways

- Data warehouse platforms vary by deployment: **appliance-based**, **cloud-native**, or **hybrid**.
- Top vendors include: **Oracle, IBM, Microsoft, Amazon, Google, Teradata, Snowflake, and Vertica**.
- These systems support features like **machine learning**, **security**, **real-time analytics**, and **multi-cloud flexibility**, enabling organizations to meet diverse data needs.

📘 Lesson 3: Selecting a Data Warehouse System

🧠 Objectives

By the end of this lesson, you will be able to:

- Identify key **criteria** for evaluating data warehouse systems
- Understand **deployment options** and how to choose between on-prem and cloud
- List and describe the **types of costs** involved in a warehouse solution

- Evaluate **support, usability, and skills** needed for implementation

---

🛠 Key Concepts

🔹 Evaluation Criteria for Choosing a Data Warehouse

1. **Location & Deployment**
    - Options: **On-premises**, **appliances**, or **cloud-based**
    - Depends on needs for:
        - **Security** (e.g., zero-trust policies, compliance with CCPA/GDPR)
        - **Speed** (fast insights and access)
        - **Geo-specific storage** for privacy regulations
2. **Features & Capabilities**
    - **Architecture**: Vendor lock-in, multi-cloud support, scalability
    - **Data Types**: Support for **structured, semi-structured**, **unstructured**, **batch**, and **streaming** data
    - **System Performance**: Ability to adapt and reoptimize over time
3. **Implementation Considerations**
    - **Data Governance**, **Migration**, **Transformation**
    - **User Management**: Secure access control and auditing
    - **Monitoring**: Real-time alerts and reporting to detect issues early
4. **Ease of Use & Skills Required**
    - Does staff have the necessary expertise?
    - How fast can new skills be acquired?
    - Complexity level of deployment
    - Role of third-party **implementation partners**
5. **Support**
    - Availability of **SLAs** (uptime, security, scalability)
    - Support **channels**: chat, email, phone, etc.
    - **Self-service tools** and **community** availability
    - Benefit of working with a **single accountable vendor**

💸 Cost Considerations

Focus on **TCO (Total Cost of Ownership)**, not just upfront price.

- **Infrastructure**: Compute, storage (cloud or on-prem)
- **Software Licensing**: Subscriptions or usage-based pricing
- **Data Migration & Integration**: Costs of onboarding and maintaining clean data
- **Administration**: Staff salaries, training, and management
- **Ongoing Support & Maintenance**: Vendor fees, updates, tuning

---

🧪 Example Use

A financial company with strict data regulations selects an **on-premises** data warehouse to meet compliance requirements but integrates cloud-based tools for visualization and reporting — balancing privacy with scalability.

🧩 Key Takeaways

- Selecting a data warehouse requires analysis of **location, architecture, data support, ease of use, vendor support**, and **costs**.
- Cloud options provide scalability and flexible pricing, but some industries may require **on-premises** for regulatory compliance.
- Always evaluate **Total Cost of Ownership** across several years, not just initial purchase costs.

📘 Lesson 4: Data Marts Overview

🧠 Objectives

By the end of this lesson, you will be able to:

- Define what a **data mart** is
- Differentiate between **data marts**, **data warehouses**, and **transactional databases**
- Describe the structure and **schema** of data marts
- Understand the **types of data marts** and how data is loaded into them

---

🛠 Key Concepts

🔹 What is a Data Mart?

- A **data mart** is a **subset** of a data warehouse, built for a **specific business function** or **user group** (e.g. sales, finance, marketing).
- Designed to support **tactical decision-making** by delivering **targeted, timely** insights.

🔹 Common Use Cases

- **Sales & Finance**: Reporting and forecasting
- **Marketing**: Customer behavior analytics
- **Operations**: Shipping, manufacturing, and warranty analysis

🔹 **Schema & Structure**

- Relational database, usually with:
  - **Star Schema**: Central fact table linked to dimension tables
  - **Snowflake Schema**: More normalized dimension tables
- Optimized for **OLAP** (read-heavy queries)

🔄 Data Mart vs Other Systems

Feature	Data Mart	Transactional DB (OLTP)	Data Warehouse (EDW)
Query Type	OLAP (read)	OLTP (write)	OLAP (read)
Data Type	Cleaned & validated	Raw operational	Cleaned & validated
Scope	Tactical, specific	Operational	Strategic, enterprise-wide
Historical Data	Yes	Rarely	Yes
Speed	Fast & lean	Fast for transactions	May be slower (larger size)

🔁 Types of Data Marts

1. **Dependent Data Mart**
   - Pulls data directly from the **enterprise data warehouse**
   - Inherits **security and governance**
   - Requires **simpler ETL pipelines**
2. **Independent Data Mart**
   - Created directly from **operational systems or external sources**
   - Needs **custom ETL**, data cleaning, and separate security setup
   - Functions like a mini data warehouse
3. **Hybrid Data Mart**
   - Combines inputs from **data warehouses** and **other external/internal sources**
   - Offers flexibility for mixed data sources

🔄 Data Pipelines for Data Marts

- **Dependent**: Leverages warehouse-cleaned data → simpler pipeline
- **Independent**: Custom **ETL processes** needed → more complex
- **Hybrid**: Mix of both pipelines depending on sources

🧩 Key Benefits

- **Speed**: Accelerated queries for faster decisions
- **Focus**: Provides only relevant data for specific use cases
- **Cost Efficiency**: Smaller scope = reduced resource demands
- **Security**: Controlled access based on business function

🧪 Example Use

A marketing team uses an independent data mart that integrates CRM and social media insights to evaluate customer sentiment trends — allowing them to rapidly adjust campaign strategies.

🧩 Key Takeaways

- A **data mart** is a targeted analytics solution, ideal for **focused business processes**.
- It stores **cleaned, historical data** in a **star or snowflake schema**.
- Data marts can be **dependent**, **independent**, or **hybrid**, each with different levels of complexity and control.
- Compared to large data warehouses, data marts are **smaller, faster**, and built for **tactical** needs.

📘 Lesson 5: IBM Db2 Warehouse

🧠 Objectives

By the end of this lesson, you will be able to:

- Describe the **key features** of IBM Db2 Warehouse
- Identify common **use cases** for Db2 Warehouse
- Understand **pipeline tool integrations** and supported environments

---

🛠 Key Concepts

🔹 What is IBM Db2 Warehouse?

- A **cloud-ready**, **client-managed** data warehouse solution that runs in:

    - **On-premises**, **cloud**, and **hybrid** environments
    - **Containerized setups** (e.g., Docker)
- Supports **Massively Parallel Processing (MPP)** for scalability and performance
- Includes **machine learning** capabilities and **in-database analytics**

⚡ Key Features

Feature	Description
Containerization	Deployable via Docker for portable and scalable deployments
BLU Acceleration	In-memory, columnar SQL processing with data skipping for faster queries
Automatic Schema Gen.	Auto-creation of schemas from raw/unstructured data
AI & ML-Ready	Includes algorithms and integrations for advanced analytics
Performance Dashboards	Monitor CPU, storage, query execution time, locks, and alerts
Spark Integration	Built-in Apache Spark cluster to run distributed analytics jobs
REST API Support	Enables running R code and analytics via API calls

---

📊 Use Cases

- **High scalability needs** (elastic workloads)
- **Hybrid / cloud / on-prem deployment flexibility**
- **Data integration** from disparate systems
- **Development of data marts** and other business analytics products
- **Handling of sensitive/regulatory data**
- **Cold data storage** for structured SQL datasets

🧩 Integrations & Tooling

🔌 Supported Languages & Tools

- **JDBC**, **Node.js**, **Spring**, **Python**, **R**, **Go**, **Apache Spark**, **Visual Studio**

📦 Open Source Resources

- GitHub: IBM DB Repository
    - Popular package: `python-ibmdb` for Python interface to Db2

🧪 Example Use

A developer builds a Docker container with **RStudio**, connects it to Db2 Warehouse, runs statistical models, and exposes the insights via a **REST API** to an internal dashboard tool.

🧩 Key Takeaways

- **IBM Db2 Warehouse** is a versatile, scalable platform for building modern data analytics environments.
- It supports **real-time, in-database analytics**, **machine learning**, and **containerized deployments**.
- Its integrations with **Spark, R, Python**, and other ecosystems make it ideal for **data science** and **BI workloads**.

📘 Lesson 6: Data Lakes Overview

🧠 Objectives

By the end of this lesson, you will be able to:

- Define what a **data lake** is
- List key **benefits** of using a data lake
- **Compare** data lakes to data warehouses

🛠 Key Concepts

🔹 What is a Data Lake?

- A **storage repository** designed to store **massive volumes** of:
  - **Structured** (e.g., databases)
  - **Semi-structured** (e.g., JSON, XML)
  - **Unstructured** (e.g., images, emails, documents) data
- Stores data in its **native/raw format**, without pre-defined schema
- Data elements are **tagged with metadata** and given **unique identifiers**

🔹 Purpose & Flexibility

- Ideal when data is generated **continuously** and used for **various future use cases**
- Often used as a **staging area** before data enters a **data warehouse or mart**
- Supports **schema-on-read**, unlike warehouses that require **schema-on-write**

🔹 Technology & Architecture

- Not tied to a specific tech — it's a **reference architecture**
- Can be implemented using:
  - **Cloud object storage** (e.g., AWS S3)
  - **Big Data frameworks** (e.g., Apache Hadoop)
  - **NoSQL systems** and RDBMS with large-scale storage capacity

---

📈 Benefits of Data Lakes

- **Stores all data types** (structured to unstructured)
- **Highly scalable**: From **terabytes to petabytes**
- **Time-efficient**: Skip upfront schema design and transformations
- **Reusable**: Raw data can serve many analytical needs
- **Future-proof**: Ready for new use cases (ML, analytics, AI)

🧪 Example Use

A retail company stores all customer interaction logs (emails, chats, web clicks) in a data lake. Later, this data is mined for training machine learning models to predict customer churn.

🧩 Data Lake vs Data Warehouse

Feature	Data Lake	Data Warehouse
Data Type	Raw, all formats	Structured, cleaned
Schema	Schema-on-read	Schema-on-write
Data Quality	May not be curated	Curated, governed
Users	Data scientists, ML engineers	Business and data analysts
Governance	Flexible, less enforced	Strict, compliant
Primary Use	Exploratory, advanced analytics, ML	Standardized reporting, BI

🧩 Vendors Supporting Data Lakes

- **Amazon**, **Google**, **Microsoft**, **IBM**, **Cloudera**, **Informatica**, **Oracle**, **Teradata**, **Snowflake**, **SAS**, **Zaloni**, and more.

🧩 Key Takeaways

- A **data lake** is ideal for storing large, varied datasets **without upfront modeling**.
- It allows **flexible, scalable**, and **future-oriented analytics**.
- It complements — rather than replaces — a **data warehouse**, and both may coexist in a modern data architecture.

📘 Lesson 7: Data Logistics & The Lakehouse Concept

🧠 Objectives

- Understand how **data architecture** mirrors **restaurant logistics**
- Compare the roles of **data lakes** and **data warehouses**
- Identify the challenges of each and why the **data lakehouse** was introduced
- Recognize the **value proposition** of combining both systems

---

🍽 Real-World Analogy: The Commercial Kitchen

- **Raw ingredients** = Incoming data
- **Loading dock** = Entry point for data
- **Pantries / Fridges** = Organized, trusted storage
- **Chefs** = Data analysts, ML engineers
- Without structure and process, cooking (analytics) becomes inefficient.

🛠 Key Concepts

🔹 Data Lakes

- Repositories for **raw**, **structured**, **semi-structured**, and **unstructured** data
- Cheap and flexible, ideal for storing large, fast-moving datasets
- Schema-on-read (structure applied later)
- Great for **staging**, **exploration**, **ML**, and **future use cases**

✅ Pros:

- Ingests diverse data quickly
- Scalable and cost-efficient

❌ Cons:

- Risk of **data swamps** (poor quality, duplication)
- Weak governance and **slow query performance**

🔹 Data Warehouses

- Optimized, structured repositories for **trusted, curated** data
- Used for **BI workloads** like dashboards and reports
- Schema-on-write (structure applied on ingestion)

✅ Pros:

- High query performance
- Strong **governance**, **data quality**, and **security**

❌ Cons:

- **Expensive** to scale
- Limited support for semi-/unstructured data
- **Slow to load** fresh data

🧪 Real-World Insight

Putting *everything* in the data warehouse is like stuffing every ingredient into a giant freezer. It's clean and efficient, but expensive and inflexible.

🔁 Enter the Data Lakehouse

- Combines the **cost-efficiency & flexibility** of data lakes
- With the **performance & structure** of data warehouses
- Supports **BI + ML** workloads from a single architecture

🔹 Value of the Lakehouse

- Store **all data types** cheaply
- Maintain **governance and metadata**
- Power **real-time AI and analytics**
- Supports **modern hybrid data ecosystems**

🧩 Key Takeaways

- Think of data systems like a restaurant: storage, organization, and flow matter.
- **Data Lakes** = raw, cheap, fast but messy
- **Data Warehouses** = organized, fast, but rigid and costly
- The **Lakehouse** = best of both worlds: **scalable**, **structured**, and **ML-ready**

Module 1 Summary: An Introduction to Data Warehouses, Data Marts, and Data Lakes

You learned that warehouse systems can exist onsite, on appliances, and on the cloud. You discovered that scalable data lakes enable organizations to provide fast, flexible data access, and serve as a self-serve staging area for machine learning development and advanced analytics, and that data marts provide specific, timely, and rapid support for making tactical decisions. You discovered that when selecting a data warehouse system, you need to consider the total cost of ownership, including infrastructure, compute and storage, data migration, and administration and data maintenance costs.

Module two

Designing, Modeling, and Implementing Data Warehouses

📘 Lesson 1: Overview of Data Warehouse Architectures

🧠 Objectives

By the end of this lesson, you will be able to:

- List **common use cases** that drive data warehouse design
- Describe the **general architecture** of a data warehouse
- Distinguish between **general and vendor-specific (reference)** architectures
- Understand IBM's **reference architecture** and its component tools

---

🛠 Key Concepts

🔹 **Use Cases Driving Data Warehouse Design**

- **Report generation** and **dashboards**
- **Exploratory data analysis**
- **Automation & machine learning**
- **Self-service analytics** for non-technical users

🔹 General Data Warehouse Architecture

This modular architecture can be customized per organization's needs.

🔧 Core Layers / Components

1. **Data Sources**
   - Flat files, databases, operational systems
2. **ETL Layer**
   - Extract, Transform, Load processes
3. **Staging / Sandbox** *(optional)*
   - Temporary storage for raw or intermediate data
   - Supports testing, transformation, and workflow development
4. **Data Warehouse Repository**
   - Central data store; typically relational databases
5. **Data Marts** *(optional)*
   - Subsets of data warehouse serving specific business functions
   - "Hub and Spoke" model when multiple marts are present
6. **Analytics & BI Layer**
   - Dashboards, reports, and data exploration tools
7. **Security**
   - Enforced throughout all layers, securing data at rest and in transit

---

🧩 Reference Architecture vs General Architecture

Aspect	General Architecture	Reference Architecture (e.g., IBM)
Flexibility	Vendor-agnostic, customizable	Tightly integrated ecosystem
Interoperability	Varies based on components used	High (components built/tested to work together)
Tooling	Open to multiple ETL/BI tools	Specific toolchain (e.g., InfoSphere, Cognos)

🏗️ IBM Reference Architecture – Layered Breakdown

Layer	Function
Data Acquisition	Pulls raw data from HR, finance, billing, etc.
Data Integration	Staging + ETL + metadata + admin tools
Data Repository	Stores transformed/integrated data (relational model)
Analytics Layer	OLAP cubes for advanced analysis
Presentation Layer	Dashboards, web portals, reports for users

---

🧰 IBM Toolchain in Reference Architecture

Tool	Role
InfoSphere DataStage	Scalable ETL with near real-time capabilities
InfoSphere Metadata Workbench	Visual data flow, lineage, and impact analysis
InfoSphere QualityStage	Data cleansing, entity resolution, data governance
IBM Db2 Warehouse	High-performance data storage (cloud & on-prem)
IBM Cognos Analytics	BI platform for reporting, dashboards, and data curation

---

🧪 Example Use

A retail company implements IBM's architecture: using DataStage for ETL, Db2 Warehouse for storage, and Cognos for dashboards showing sales KPIs. QualityStage ensures product names and customer data are consistent and trustworthy across systems.

🧩 Key Takeaways

- A **general EDW architecture** includes data sources, ETL, staging, warehouse repository, data marts, and analytics tools.
- **Reference architectures** (like IBM's) build on this model with tightly integrated components that improve **interoperability**, **scalability**, and **governance**.
- IBM's data warehouse stack combines **InfoSphere**, **Db2**, and **Cognos** to deliver an end-to-end solution for enterprise analytics.

📘 Lesson 2: Cubes, Rollups, and Materialized Views

🧠 Objectives

By the end of this lesson, you will be able to:

- Explain the concept of a **data cube** in OLAP systems
- Apply operations like **slice**, **dice**, **pivot**, **drill**, and **roll-up**
- Define and understand the use of **materialized views**
- Recall **use cases** and **SQL examples** for materialized views in Oracle, PostgreSQL, and Db2

🛠 Key Concepts

🔹 Data Cubes in OLAP

- Represent **multidimensional data** from a **star or snowflake schema**
- Dimensions = coordinates (e.g., Product, Year, State)
- Fact = metric in the cell (e.g., total sales in $K)

🔹 OLAP Cube Operations

Operation	Description
Slice	Fix one dimension → reduces the cube by one axis (e.g., sales in 2018 only)
Dice	Select subset of values in multiple dimensions (e.g., just T-shirts and Jeans)
Drill Down	Navigate deeper into hierarchies (e.g., T-shirts → Slim fit, Classic)
Drill Up	Reverse of drill down (back to broader category)
Pivot	Rotate cube axes (e.g., swap Product and Year axes)
Roll Up	Aggregate along a dimension (e.g., total sales per region or average price)

🧩 Materialized Views (MVs)

🔸 What is a Materialized View?

- A **snapshot** or **cached query result** that is stored and **can be queried**
- Improves **performance** by avoiding recomputation of expensive queries
- Can be used for:
  - **Data replication** in staging environments
  - **Pre-computed joins/aggregations** for faster analytics

🔸 Refresh Strategies

Type	Description
Never	Only populated once on creation
On Demand	Refreshed manually (e.g., daily or after data loads)
Immediate	Auto-refresh after data changes

🧪 SQL Examples

✅ Oracle

```
CREATE MATERIALIZED VIEW my_mat_view
REFRESH FAST START WITH SYSDATE NEXT SYSDATE + 1
AS SELECT * FROM my_table_name;
```

✅ PostgreSQL

```
CREATE MATERIALIZED VIEW my_mat_view
TABLESPACE tablespace_name AS
SELECT * FROM table_name;

-- Refresh manually:
REFRESH MATERIALIZED VIEW my_mat_view;
```

✅ Db2 (Materialized Query Tables - MQT)

```
CREATE TABLE emp AS (
  SELECT e.name, d.dept_name
  FROM Employee e, Department d
  WHERE e.dept_id = d.dept_id
)
DATA INITIALLY DEFERRED
REFRESH IMMEDIATE
ENABLE QUERY OPTIMIZATION;
```

🧩 Key Takeaways

- A **data cube** is a powerful multidimensional model used for OLAP queries, with flexible operations like **slicing**, **dicing**, **pivoting**, and **rolling up**.
- **Materialized views** store precomputed query results for **performance optimization**, and can be configured with various refresh policies.
- **Oracle, PostgreSQL, and Db2** all support materialized views (called MQTs in Db2), with different syntax and refresh capabilities.

📘 Lesson 3: Grouping Sets in SQL

🧠 Objective

Understand how to use **GROUPING SETS** in SQL to perform **multi-level aggregations** in a single query — a powerful tool for building **OLAP-style summaries**, such as those found in **data cubes**.

🛠 Key Concepts

🔹 What Are Grouping Sets?

- `GROUPING SETS` allow you to define **multiple GROUP BY clauses** in a single SQL query.
- Think of them as a way to **create subtotals and totals** without writing multiple queries or complex unions.
- It is the **foundation** of SQL operations like `CUBE` and `ROLLUP`.

🧪 Example: Sales Summary

Suppose you have a table:

```
sales(product, region, year, total_sales)
```

You want:

- Sales per `product`
- Sales per `region`
- Sales per `(product, region)`
- A **grand total**

Using `GROUPING SETS`:

```
SELECT
  product,
  region,
  SUM(total_sales) AS total_sales
FROM sales
GROUP BY GROUPING SETS (
  (product),
  (region),
  (product, region),
  ()
);
```

📊 What the Query Produces

product	region	total_sales
T-shirt	NULL	5000
NULL	East	7000
T-shirt	East	3000
NULL	NULL	15000

- `(product)` → subtotal by product
- `(region)` → subtotal by region

- `(product, region)` → detailed aggregation
- `()` → grand total

🔁 Related: ROLLUP vs CUBE vs GROUPING SETS

Syntax	Use Case
`ROLLUP`	Hierarchical summaries (e.g., product → total)
`CUBE`	All possible combinations of dimensions
`GROUPING SETS`	Full manual control over which groups to aggregate

Example:

```
GROUP BY ROLLUP (region, product) -- Implicit grouping sets: (region, product), (region), ()
GROUP BY CUBE (region, product)   -- All: (region, product), (region), (product), ()
```

🧩 Key Takeaways

- `GROUPING SETS` give you fine-grained control over **multi-level aggregations**.
- Ideal for **reporting, dashboards**, and **data mart summaries**.
- Can replace multiple `UNION ALL` statements with a **single efficient query**.
- Foundational for **OLAP**-style SQL extensions like `ROLLUP` and `CUBE`.

📘 Lesson 4: Facts and Dimensional Modeling

🧠 Objectives

By the end of this lesson, you will be able to:

- Define **facts** and **fact tables**
- Define **dimensions** and **dimension tables**
- Understand how facts and dimensions relate in data warehousing
- Describe real-world examples of fact-dimension modeling

🛠 Key Concepts

🔹 What Are Facts?

- **Facts** are the measurable data in a business process.
- Typically **quantitative**: e.g., sales amount, number of items sold, temperature

- Can also be **qualitative**: e.g., weather conditions ("partly cloudy")

🔸 **Fact Table**

- Stores the **core metrics** of business activity
- Contains **foreign keys** referencing dimension tables
- Can be:
  - **Transaction-level** (e.g., every sale)
  - **Aggregated/Summary** (e.g., weekly sales totals)
  - **Accumulating Snapshots** (track process lifecycle — e.g., order to shipment)

🍼 **Example: Computer Order Lifecycle**

Order ID	Order Date	Payment Date	Build Start	Build End	Ship Date
1001	Jan 5	Jan 6	Jan 7	Jan 10	Jan 11

🔹 **What Are Dimensions?**

- **Dimensions** are **categorical variables** that describe or contextualize facts
- Used in **filtering**, **grouping**, and **labeling** in analysis

🔸 **Dimension Table**

- Stores attributes for each dimension entity
- Common types:
  - **Product**: make, model, category
  - **Employee**: name, department, title
  - **Time**: day, week, quarter
  - **Location**: country, state, city

🧩 Fact-Dimension Relationship

📊 **Example Schema: Car Dealership Sales**

Table	Contents
Sales	Fact table with: Sale ID (PK), Sale Date, Sale Amount, FK: Vehicle ID, Salesperson ID
Vehicle	Dimension table with: Vehicle ID (PK), Make, Model
Salesperson	Dimension table with: Salesperson ID (PK), Name, Dept

- **Fact Table** connects to multiple **Dimension Tables** via **foreign keys**

🧪 Real-World Example

"Sale amount = $20,000" is just a number — until we attach:

- **Date**: July 12
- **Location**: Chicago
- **Vehicle**: Toyota Camry
- **Salesperson**: Jane Doe

Now it becomes a **complete, analyzable business event**.

🧩 Key Takeaways

- **Facts** are measurable business events (quantitative or qualitative).
- **Dimensions** categorize and give context to those facts.
- **Fact tables** link to **dimension tables** via **foreign keys**.
- Dimensional modeling is central to OLAP systems and star/snowflake schemas.

📘 Lesson 5: Data Modeling Using Star and Snowflake Schemas

🧠 Objectives

By the end of this lesson, you will be able to:

- Describe **star schema** modeling with facts and dimensions
- Define the **snowflake schema** as a normalized extension of the star schema
- Compare **normalization** levels between the two models
- Apply schema modeling to real-world business processes

🛠 Key Concepts

🔹 Star Schema

- A **central fact table** surrounded by **dimension tables**
- Represents a **hub-and-spoke** model
- Commonly used in **data marts** for analytics

🔸 Fact Table

- Stores **quantitative facts** (e.g., sales amount, tax, discount)
- Contains **foreign keys** linking to dimensions

- Grain = level of detail (e.g., individual sales line items)

🔸 **Dimension Tables**

- Contain **descriptive attributes** (e.g., date, product, store, cashier)
- Used for **filtering**, **grouping**, **labeling** in analysis
- Linked to fact table via **primary–foreign key relationships**

🧪 Example: A to Z Discount Warehouse POS Model

Table	Attributes
Fact: POS	POS ID, Amount, Quantity, Tax, Discount, FK: StoreID, ProductID, DateID, CashierID, MemberID
Store	StoreID, Store Name, FK: CityID
Product	ProductID, Name, Brand, Category
Date	DateID, Full Date, Day, Month, Quarter, Year
Cashier	CashierID, First Name, Last Name
Member	MemberID, Customer Info

❄️ Snowflake Schema

- **Normalized** version of star schema
- Dimension tables are split into **hierarchical sub-tables**
- Reduces data redundancy and **storage footprint**

🧩 Example Normalizations

- **Store table** → references **City table**, which references **State**, which references **Country**
- **Product table** → references **Brand table** and **Category table**
- **Date table** → split into **Day**, **Month**, **Quarter** dimension tables

---

🔁 Star vs. Snowflake Comparison

Feature	Star Schema	Snowflake Schema
Normalization	Denormalized	Normalized (at least one dimension)
Performance	Faster for querying	Slightly slower (more joins)
Storage	Larger	More efficient
Complexity	Simple design	More complex, hierarchical
Use Case	Data marts, quick access	Enterprise DW with large hierarchies

---

🧩 Key Takeaways

- A **star schema** provides simplicity and speed for analytical queries.
- A **snowflake schema** is a normalized extension for **storage optimization** and **hierarchical data**.
- Schema design starts with:
    1. **Choosing a business process**
    2. **Defining the grain (granularity)**
    3. **Identifying dimensions**
    4. **Identifying facts**
- Fact and dimension tables are joined by **primary–foreign key** relationships, forming the backbone of both schema types.

📘 Lesson 6: What are SCDs?

Slowly Changing Dimensions (SCDs) are techniques in data warehousing used to **track changes** in dimension data (like customer info, product details) **over time**, ensuring accurate historical reporting and data integrity.

📊 Types of SCDs (with examples)

Type	Strategy	Tracks History?	Example
Type 0	Keep original value only	❌ No	Product code remains unchanged
Type 1	Overwrite old data	❌ No	Update email, old value lost
Type 2	Add new row with versioning	✅ Yes	New row for each address change
Type 3	Add new column for limited history	⚠️ Limited	Store current and previous address in columns
Type 4	Use a separate historical table	✅ Yes	One table for current, another for history
Type 6	Hybrid (Type 1 + 2 + 3)	✅ Full + comparisons	Row versioning + current/previous columns

---

🛠️ Key Considerations

- Choose the SCD type **based on business needs** (e.g., compliance may require full history).
- Type 2 and 6 need **careful versioning logic** (start date, end date, current flag).
- Consider **storage impact and query performance**, especially for Type 2 and 6.

- Design your **ETL process** to match the SCD type (e.g., insert vs. update logic).

✅ In Summary

SCDs help track how data changes over time.
Use:

- **Type 1** for simplicity,
- **Type 2** for full history,
- **Type 6** for flexible hybrid tracking.

Understanding these types ensures reliable, insightful reporting in a data warehouse.

📘 Lesson 7: Why Use Star and Snowflake Schemas?

⭐ ❄️

Both schemas organize **fact and dimension tables** in a data warehouse, but differ in **structure, performance, and use case**.

📊 Star vs. Snowflake Schema Comparison

Attribute	Star Schema	Snowflake Schema
Structure	Denormalized	Normalized (dimensions split into subtables)
Read Performance	Fast (fewer joins)	Slower (more joins)
Write Performance	Moderate	Fast
Storage Space	Moderate to high	Low to moderate
Query Complexity	Simple	Complex
Best For	OLAP / Reporting (Data Marts)	OLTP / Transactional Warehousing

---

🧠 Key Concepts

- **Normalization** reduces redundancy by storing dimension hierarchies in separate tables (snowflake).
- **Star schema** flattens dimensions into single tables for better read performance.
- **Surrogate keys** (usually integers) are used in fact tables to save space and speed up joins.

🛠️ When to Use What

- Use **Star Schema** when:
  - You prioritize **read performance** and **simplicity**.
  - You're building **data marts** or **analytical dashboards**.
- Use **Snowflake Schema** when:
  - You prioritize **write efficiency** and **storage savings**.
  - You're working in **transaction-heavy systems**.

⚖️ Trade-Offs & Hybrid Approach

- You can **mix both**: store data in snowflake form but create **materialized views** to simulate a star schema for analysis.
- Avoid over-normalizing rarely-used fields (like month names); calculate them on demand with SQL functions instead.

🧪 Scenario Summary

If you're handed a large denormalized dataset:

- You can apply either schema using **surrogate keys** to reduce fact table size.
- Your final choice depends on:
  - Data size and growth rate
  - Query needs
  - Business requirements

✅ Conclusion

- **Star = Simple + Fast Reads**
- **Snowflake = Compact + Fast Writes**

- **Choose based on workload: OLAP (Star), OLTP (Snowflake), or Hybrid for balance**

📘 Lesson 8: Staging Areas for Data Warehouses

🧠 Objectives

By the end of this lesson, you will be able to:

- Define what a **data warehouse staging area** is
- Explain why staging areas are used in **ETL workflows**
- Describe how staging enables **data integration** from multiple sources

---

🛠 Key Concepts

🔹 What is a Staging Area?

- A **temporary or intermediate storage** layer used in the **ETL (Extract, Transform, Load)** process
- Acts as a **bridge** between **source systems** and **target repositories** (e.g., data warehouses, data marts)
- Can be **transient** (erased after ETL) or **persistent** (for archival or debugging)

🔧 Common Implementations

- **Flat files** (e.g., `.csv`, `.txt`) managed via **Bash**, **Python**, etc.
- **Staging tables** in relational DBs (e.g., **Db2**)
- **Self-contained staging databases** within BI tools (e.g., **Cognos Analytics**)

🧪 Example Architecture: Cost Accounting OLAP System

1. **Data Sources**:
   - Payroll, Sales, and Purchasing OLTP systems
2. **Staging Area**:
   - Data extracted to **Staging Tables** in a Staging Database
   - Transformed via SQL (e.g., type formatting, cleansing)
3. **Integration**:
   - Data joined and conformed across departments
4. **Loading**:
   - Final clean data loaded into **Cost Accounting** data warehouse

---

🔄 Functions of the Staging Area

Function	Description
Integration	Combine data from multiple source systems
Change Detection	Capture new or updated records
Scheduling	Run ETL steps in sequence, on schedule, or in parallel
Cleansing/Validation	Remove duplicates, fill in missing values
Aggregation	Convert detailed data into summaries (e.g., daily → monthly)
Normalization	Standardize naming conventions, data types, and codes

---

🛡 Benefits of Staging

- **Decouples ETL** from source systems → reduces risk of affecting operational data
- Simplifies construction and **maintenance** of ETL pipelines
- Enables **recovery** if something goes wrong in the pipeline
- Can retain data for **archiving** or **debugging** purposes

---

🧩 Key Takeaways

- A **staging area** is a core part of ETL architecture, enabling reliable and flexible data movement.
- It helps **integrate**, **clean**, **transform**, and **standardize** data before it hits the warehouse.
- While often **temporary**, staging areas can be extended to serve **audit** or **troubleshooting** needs.

📘 Lesson 9: Verifying Data Quality

🧠 Objectives

By the end of this lesson, you will be able to:

- Define **data quality verification**
- Explain why organizations need to verify data quality
- Identify common **data quality issues**
- Outline a process for detecting and handling **bad data**
- Recognize **enterprise tools** used for data verification

🛠 Key Concepts

🔹 What is Data Quality Verification?

- The process of checking data for:
  - **Accuracy**: Is the data correct?
  - **Completeness**: Is anything missing?
  - **Consistency**: Is data entered in a standardized format?
  - **Currency**: Is the data up-to-date?
- Ensures **reliable, integrated, and analyzable** data
- Enables **confidence in decision-making**, analytics, and machine learning

---

📉 Common Data Quality Issues

Concern	Example
Accuracy	Typos, duplicated records, misaligned CSVs
Completeness	Nulls, missing rows, placeholders like “999”
Consistency	Inconsistent date formats, units (e.g., kg vs lbs), naming variations (e.g., "Mr. John Doe" vs "John Doe")
Currency	Outdated addresses or names, old dimension records

🔄 The Data Quality Management Process

1. **Detect Issues**
   - Write **SQL rules** to identify nulls, duplicates, invalid values
2. **Quarantine / Flag** bad data
3. **Report** issues to relevant domain experts
4. **Investigate Lineage** to find upstream causes
5. **Correct & Clean** data (manually or through scripts)
6. **Automate** the workflow using nightly validation scripts
7. **Review** unresolved issues through generated reports

🧪 Example: Staging Area Validation

- During nightly ETL loads, a validation script:
  - Flags rows with missing, duplicated, or invalid data
  - Applies transformation rules to clean known issues
  - Logs remaining errors for manual review

🧰 Enterprise Data Quality Tools

Vendor	Tool Name
IBM	InfoSphere Information Server for Data Quality
Informatica	Informatica Data Quality
SAP	Data Quality Management
SAS	SAS Data Quality
Talend	Open Studio for Data Quality
Precisely	Spectrum Quality
Microsoft	Data Quality Services
Oracle	Enterprise Data Quality
Open Source	OpenRefine

🧩 Spotlight: IBM InfoSphere for Data Quality

- Monitors, cleans, and standardizes data
- Maintains **data lineage** and supports **continuous monitoring**
- Unified interface to explore, validate, and trust your data

---

🧩 Key Takeaways

- **Data quality verification** is essential for reliable analytics and modeling
- Common issues include: **missing data**, **typos**, **inconsistencies**, and **outdated records**
- Organizations should build **automated data cleaning pipelines**
- Enterprise tools like **IBM InfoSphere** support end-to-end data quality workflows

---

📘 Lesson 10: Populating a Data Warehouse

🧠 Objectives

By the end of this lesson, you will be able to:

- Describe **populating a data warehouse** as an ongoing ETL process
- List the **main steps** for initial and incremental data loading
- Understand methods for **change detection**
- Create and populate tables for a **star schema** manually
- Recall the **maintenance practices** needed to keep a data warehouse efficient

---

🛠 Key Concepts

🔹 Populating a Data Warehouse

- Begins with an **initial load** (one-time setup)
- Followed by **ongoing incremental loads** (daily, weekly, etc.)
- Occasionally requires **full refreshes** (e.g., major schema changes or recovery)

🧱 Initial Load Steps

1. Ensure:
    - Schema is modeled
    - Staging data is available
    - Data is verified for quality
2. **Instantiate schema** and create **fact and dimension tables**
3. **Establish relationships** using foreign key constraints
4. **Insert transformed & cleaned data** from staging area into tables

---

🔁 Incremental Loads

- Load **new or changed data** using ETL jobs or automation tools
- Methods for change detection:
    - **Timestamps** (created_at / modified_at fields)
    - **CDC (Change Data Capture)** mechanisms in RDBMS
    - **Brute-force diff** between source and warehouse if volume is low

---

⚙️ Tools & Automation

- **Db2 Load Utility** (bulk loading)
- **Apache Airflow**, **Apache Kafka** (workflow orchestration)
- **Cron jobs**, **Python**, **SQL**, **Bash scripts** (custom automation)
- **IBM InfoSphere DataStage** (enterprise ETL)

🧪 Example: Manual Load for a Star Schema

Sample Schema: `sales`

1. **Dimension Tables**
    - `sales.DimSalesPerson` → Columns: SalesPersonID (PK), AltID, Name

- Create using `CREATE TABLE`, then populate with `INSERT INTO`

2. **Fact Table**
   - `sales.FactAutoSales` → Columns: TransactionID (PK), SalesID, Amount, SalesPersonID, AutoClassID, SalesDateKey
   - Add foreign keys using `ALTER TABLE ... ADD CONSTRAINT ... REFERENCES`
3. **Populate Fact Table**

```
INSERT INTO sales.FactAutoSales (SalesID, Amount,
SalesPersonID, AutoClassID, SalesDateKey)
VALUES (1629, 42000, 2, 1, 4);
```

🧹 Maintenance Tasks

- **Schedule incremental loads** (daily/weekly)
- **Archive or delete old data** (monthly/yearly) to reduce warehouse size
- Store older data in **slower, cheaper storage**

🧩 Key Takeaways

- Populating a data warehouse includes:
  - **Initial setup** of schema and relationships
  - **ETL data loading** from staged sources
  - **Ongoing incremental loads** to keep data current
- **Fact tables** change frequently; **dimension tables** are relatively static
- Automation is key for **efficiency, reliability, and scalability**
- Maintenance ensures long-term performance and cost-effectiveness

---

Module two

🎉 Final Summary: IBM Data Warehouse Fundamentals

🧠 Final Review of Core Concepts

🏛️ What Is a Data Warehouse?

- A **data warehouse** is a centralized system that **aggregates data from multiple sources** to support analytics and decision-making.
- Common vendors: **IBM**, **Teradata**, **Oracle**, **Microsoft**, **Google BigQuery**, **Snowflake**, **Amazon Redshift**.

🧱 Architecture Components

- **Data Sources → ETL Pipelines → Staging/Sandbox Areas → Enterprise Data Warehouse (EDW)** → (Optional) **Data Marts**
- **Data Marts**: Serve specialized business units (e.g., sales, marketing).
- **Data Lakes**: Store **raw**, unstructured or semi-structured data with **metadata tagging**—ideal for flexible, undefined use cases.

🧩 Facts & Dimensions

- **Facts**: Numerical values (e.g., sales amount, temperature)
- **Dimensions**: Provide context (e.g., time, location, salesperson)
- Used in **fact tables** and **dimension tables** connected via **foreign and primary keys**

🌟 Star vs. Snowflake Schema

Feature	Star Schema	Snowflake Schema
Structure	Flat, denormalized	Normalized dimension hierarchies
Performance	Faster for querying	Efficient for storage & integrity
Use Case	Simpler, OLAP-focused systems	More complex dimensions, larger scale

---

📊 Data Cubes

- **Data Cube** = Multidimensional representation of a fact table
- Operations:
  - **Drill-down** (more detail)
  - **Roll-up** (summary)
  - **Slice/Dice** (filter combinations)

📦 Materialized Views

- **Precomputed snapshot** of a query
- Improves performance for:
  - Heavy joins
  - Aggregations
  - Repeated reports
- Can be **refreshed** on-demand or by schedule

🔄 Populating the Warehouse

- **Initial Load** → All tables and relations created & filled
- **Incremental Load** → Periodic updates (daily/weekly) using:
  - Timestamps
  - Change-detection logic
  - Scheduled ETL jobs (e.g., with Apache Airflow, Bash, DataStage)

🧼 Data Quality & Verification

- Check for:
  - **Accuracy**
  - **Completeness**
  - **Consistency**
  - **Currency (freshness)**
- Use rules, validation scripts, and tools like:
  - **IBM InfoSphere**
  - **Informatica**
  - **OpenRefine**

🎓 What's Next?

- You now have a **solid foundation** in data warehousing.
- Showcase your skills via the practice projects & final labs.
- Continue learning via:
  - 🔹 *IBM Data Warehouse Engineer Certificate*
  - 🔹 *IBM Data Engineering Certificate*
  - 🔹 *BI Analyst Certificate*
  - 🔹 *BI Foundations: SQL, ETL, DW*

⏳ Estimated completion: 3–6 months
📚 Check the links in the course conclusion section!

💡 Final Takeaway

Your journey into data warehousing is just beginning. With the right tools and practice, you'll be designing, querying, and optimizing enterprise-level systems like a pro! 👨‍💻📈